Cubic Spline Interpolation

Ori Gonen

1 Introduction

The goal of this assignment is to implement cubic spline interpolation. Given a set of points (x_i, y_i), we construct a smooth function $S(x)$ that interpolates the data and is twice continuously differentiable.

2 Mathematical Background

On each interval $[x_i, x_{i+1}]$, the spline is a cubic polynomial:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Its derivatives are:

$$S_i'(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2$$
$$S_i''(x) = 2c_i + 6d_i(x - x_i)$$

Conditions

- Interpolation:
$$S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}$$
- First derivative continuity:
$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$$
- Second derivative continuity:
$$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1})$$
- Natural boundary conditions:
$$S''(x_1) = 0, \quad S''(x_n) = 0$$

We get a linear system with $4(n-1)$ variables (the variables are the coefficients) and equations.

2.1 Derivation of the Linear System

We construct the spline on each interval $[x_i, x_{i+1}]$ as:

$$\begin{aligned} S_i(x) &= a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \\ S_i'(x) &= b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2 \\ S_i''(x) &= 2c_i + 6d_i(x - x_i) \end{aligned}$$

Natural boundary conditions At the left endpoint:

$$\begin{aligned} S''(x_1) &= 0 \\ 2c_1 + 6d_1(x_1 - x_1) &= 0 \\ c_1 &= 0 \end{aligned}$$

At the right endpoint:

$$\begin{aligned} S''(x_n) &= 0 \\ 2c_{n-1} + 6d_{n-1}(x_n - x_{n-1}) &= 0 \\ c_{n-1} + 3d_{n-1}(x_n - x_{n-1}) &= 0 \end{aligned}$$

Interpolation conditions For $i = 1, \ldots, n-1$:

$$\begin{aligned} S_i(x_i) = y_i \quad &\Rightarrow \quad a_i = y_i \\ S_i(x_{i+1}) = y_{i+1} & \end{aligned}$$

Let $h_i = x_{i+1} - x_i$. Then:

$$a_i + b_i h_i + c_i h_i^2 + d_i h_i^3 = y_{i+1}$$

First derivative continuity For $i = 1, \ldots, n-2$:

$$\begin{aligned} S_i'(x_{i+1}) &= S_{i+1}'(x_{i+1}) \\ b_i + 2c_i h_i + 3d_i h_i^2 &= b_{i+1} \\ b_i + 2c_i h_i + 3d_i h_i^2 - b_{i+1} &= 0 \end{aligned}$$

Second derivative continuity For $i = 1, \ldots, n-2$:

$$\begin{aligned} S_i''(x_{i+1}) &= S_{i+1}''(x_{i+1}) \\ 2c_i + 6d_i h_i &= 2c_{i+1} \\ c_i + 3d_i h_i - c_{i+1} &= 0 \end{aligned}$$

Total number of equations We obtain:

- 2 boundary equations
- $n-1$ interpolation equations at left point
- $n-1$ interpolation equations at right point
- $n-2$ first derivative continuity equations
- $n-2$ second derivative continuity equations

Thus the total number of equations is

$$2 + (n-1) + (n-1) + (n-2) + (n-2) = 4(n-1)$$

This is also the number of coefficients! So we can solve the system.

3 Implementation

3.1 Data Structure

We define a struct:

```
struct Zlepek
    x::Vector{Float64}
    y::Vector{Float64}
    coefficients::Vector{Vector{Float64}}
end
```

Each interval has coefficients (a_i, b_i, c_i, d_i).

3.2 Building the System

construct a linear system:

$$A\mathbf{u} = \mathbf{b}$$

where $\mathbf{u}$ is a vector of length $4(n-1)$ which has all the coefficients.
We then solve by Julia's backslash operator. The solution vector is then reshaped into per-interval coefficients. (vector of $4(n-1)$ floats $\longrightarrow$ vector of $n-1$ vectors of 4 coefficients)

4 Evaluation

To evaluate the spline at a point x, we:

- Find the interval index i where $x \in [x_i, x_{i+1}]$.
- Evaluate with the corresponding coefficients

5 Testing

We verify correctness using multiple functions:

- Linear function
- Constant
- sine
- sine with uneven spacing

6 Results

The implementation successfully:

- Interpolates all given points
- Produces smooth curves

Different example functions (sin, random point, Gaussian) are presented in demo file, and can be seen in Figure 1.

Downfall: undersampling (i.e., points are too far apart for "high frequency" functions) results in interpolation which is not representative of the original function. As we can see in Figure 2, not sampling sufficiently frequent will result in unrepresentative interpolations. In the top row,

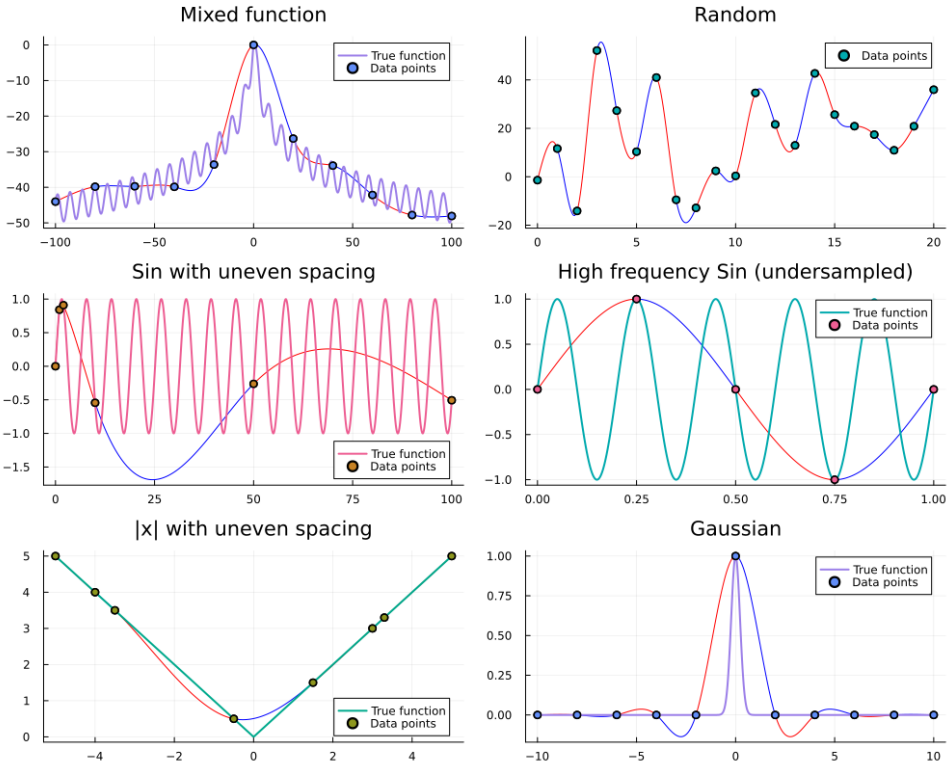


Figure 1: Demonstrations with different functions

we sample frequent enough to not tell the difference. however, on the bottom we see problems: in the left we see we always sample on 0s of the function, so the interpolation is constant 0. on the right, we sample so slow that it looks like we are sampling a lower frequency sin function.

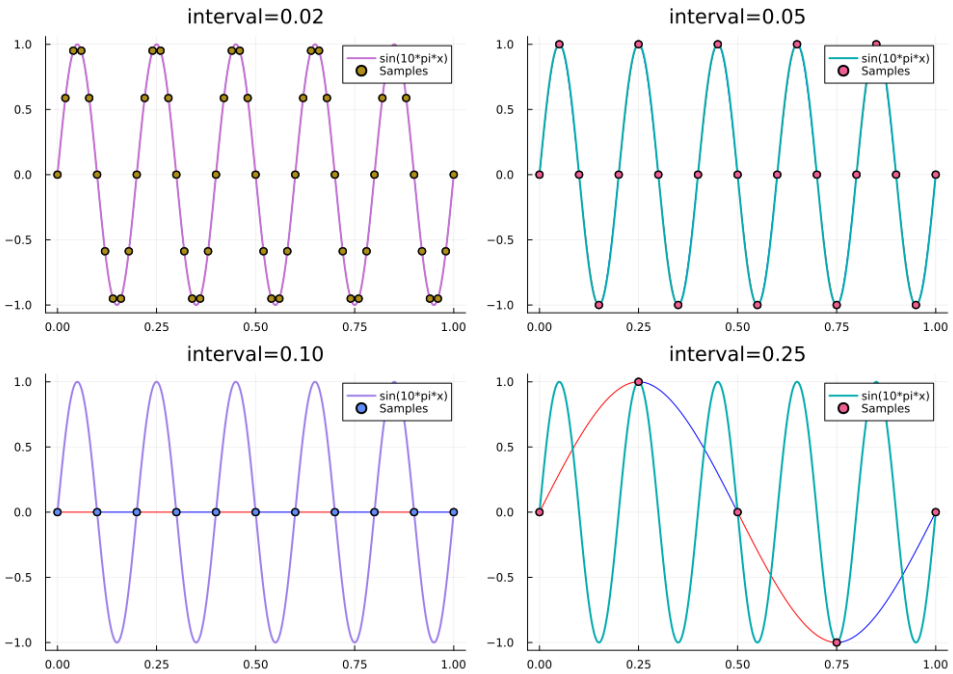


Figure 2: Cubic spline interpolation for Sin function with different sampling frequencies